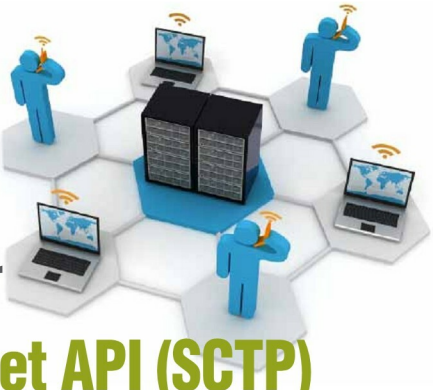

Creating Your Own Server
The Socket API (SCTP)

This article on socket programming deals with the Stream Control Transmission Protocol (SCTP).

Part—5

Similar to TCP and UDP, SCTP provides some features of both. It is message-oriented, and provides a reliable sequenced delivery of messages. SCTP supports multi-homing, i.e., multiple IPs on both sides of the connection. So it is called an association instead of a connection, as a connection involves communication between two IPs, while an association refers to communication between two systems that may have multiple IPs.

SCTP can provide multiple streams between connection endpoints, and each stream will have its own reliable sequenced delivery of messages, so any lost message will not block the delivery of messages in any of the other streams. SCTP is not vulnerable to SYN flooding, as it requires a 4-way handshake.

We will discuss the science later, and now jump to the code, as we usually do. But first you need to know the types of SCTP sockets:

- A one-to-one socket that corresponds to exactly one SCTP association (similar to TCP).
- A one-to-many socket, where many SCTP associations can be active on a socket simultaneously (similar to UDP receiving datagrams from several endpoints).

One-to-one sockets (also called TCP-style sockets) were developed to ease porting existing TCP applications to SCTP, so the difference between a server implemented using TCP and SCTP is not much. We just need to modify the call to *socket()* to *socket(AF_INET, SOCK_STREAM, and IPPROTO_SCTP)* while everything else stays the same—the calls to *listen(), accept()* for the server, and *connect()* for the client, with *read()* and *write()* calls for both.

Now let's jump to the one-to-many or UDP-style socket, and write a server using multiple streams that the client follows. The *smtpserver.c* and *smtpclient.c* files can be found at http://linuxforu.com/article_source_code/dec11/socketapi-part5.zip.

```
#include <stdio.h>
#include <string.h>
#include <time.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <netinet/in.h>
#include <netinet/sctp.h>
#include <arpa/inet.h>

#define MAX_BUFFER   1024

int main() {
      int sfd, cfd, len, i;
      struct sockaddr_in saddr, caddr;
      struct sctp_initmsg initmsg;
      char buff[INET_ADDRSTRLEN];
```